



黑龙江工业学院

毕业设计

题 目： 基于 YOLOv8 与注意力机制的
疲劳驾驶面部识别系统

学 院： 电气与信息工程学院

专 业： 人工智能

班 级： 22 人工智能 1 班

作者姓名：

学 号：

研究方向：

指导教师：

2026 年 5 月



黑龙江工业学院学士学位论文原创性声明

本人郑重声明：所呈交的学士学位论文，是本人在指导教师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的作品成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律结果由本人承担。

学位论文作者签名：

日期：2026 年 月 日



摘要

疲劳驾驶是道路交通事故的重要诱因之一。驾驶员长时间驾车或睡眠不足时，容易出现闭眼时间延长、眨眼频繁、打哈欠增多等面部状态变化。相比接触式生理信号和车辆行为检测方法，视觉检测具有非接触、成本低、便于部署的特点，但传统视觉方法依赖人工阈值，在光照变化、姿态偏转和局部遮挡情况下鲁棒性不足。

针对上述问题，本文设计并验证了一种基于 YOLOv8 和 CBAM 注意力机制的疲劳驾驶面部识别方法。方法以驾驶员面部图像为输入，检测闭眼、睁眼和打哈欠三类目标；以 YOLOv8n 为基线模型，在 Backbone 末端与 Neck 输入端之间嵌入 CBAM 模块，以增强眼部、嘴部局部特征表达。实验数据采用 YOLO 格式标注，并完成数据划分、标注解析和输入预处理。

实验以 YOLOv8n 和 YOLOv8n+CBAM 进行对比，评价指标包括 Precision、Recall、mAP50、mAP50-95、类别指标和推理耗时。结果表明，引入 CBAM 后，模型 mAP50 由 0.8347 提高到 0.8580，mAP50-95 由 0.5368 提高到 0.5470，Recall 由 0.9114 提高到 0.9370，Yawn 类别 mAP50 由约 0.693 提高到 0.762。

同时，本文基于 Python、PyTorch、OpenCV 和 PyQt5 实现桌面演示系统，支持模型加载、图片和视频检测、疲劳状态判定、结果显示、截图保存和日志记录。研究表明，该方法能够形成从数据处理、模型训练到系统演示的完整实验流程。

关键词：疲劳驾驶检测；YOLOv8；CBAM 注意力机制



Abstract

Driver fatigue is an important cause of road traffic accidents. When drivers keep driving for a long time or lack sleep, their facial states may show prolonged eye closure, frequent blinking and yawning. Compared with contact physiological signals and vehicle-behavior methods, visual detection is non-contact, low-cost and easier to deploy, but traditional visual methods depend on manual thresholds and are sensitive to illumination, pose and occlusion.

To address these problems, this thesis designs and verifies a driver fatigue facial recognition method based on YOLOv8 and the CBAM attention mechanism. The method detects three targets: closed eyes, open eyes and yawning. YOLOv8n is used as the baseline model, and CBAM is embedded between the end of the Backbone and the input of the Neck to enhance local features around the eyes and mouth.

Comparative experiments are conducted between YOLOv8n and YOLOv8n+CBAM. Precision, Recall, mAP50, mAP50-95, class-level indicators and inference time are used for evaluation. After CBAM is introduced, mAP50 increases from 0.8347 to 0.8580, mAP50-95 from 0.5368 to 0.5470, Recall from 0.9114 to 0.9370, and the mAP50 of the Yawn class from about 0.693 to 0.762.

A desktop demonstration system is also implemented with Python, PyTorch, OpenCV and PyQt5. It supports model loading, image and video detection, fatigue-state judgment, result display, screenshot saving and log recording, forming a complete experimental workflow from data processing and model training to system demonstration.

Key words: driver fatigue detection; YOLOv8; CBAM attention mechanism

目录

摘要	2
Abstract	3
第 1 章 绪论	6
1.1 研究背景	6
1.2 研究意义	7
1.3 国内外研究现状	7
1.4 本文主要研究内容	8
1.5 论文组织结构	9
第 2 章 相关技术理论基础	10
2.1 YOLOv8 目标检测算法	10
2.2 CBAM 注意力机制	10
2.3 PyTorch 与 Ultralytics 框架	11
2.4 OpenCV 图像处理技术	11
2.5 PyQt5 桌面界面技术	11
2.6 模型运行与可视化基础	12
2.7 本章小结	12
第 3 章 数据集构建与预处理	13
3.1 数据来源与采集场景说明	13
3.2 标注格式与数据集划分	13
3.3 类别分布统计与样本不均衡分析	14
3.4 数据预处理流程	15
3.5 数据集不足与影响分析	15
3.6 本章小结	16
第 4 章 模型/算法设计与改进	17
4.1 疲劳面部特征检测任务分析	17
4.2 YOLOv8n 基线模型设计	17
4.3 CBAM 注意力模块嵌入方案	18
4.4 疲劳状态判定算法	18
4.5 训练策略与参数设置	19
4.6 本章小结	20
第 5 章 实验设计与结果分析	21



5.1 实验环境与评价指标.....	21
5.2 两组对比实验设计.....	21
5.3 PR 曲线与 F1 曲线分析.....	22
5.4 类别结果与混淆矩阵分析.....	25
5.5 系统演示结果分析.....	26
5.6 本章小结	28
第 6 章 系统设计与实现.....	29
6.1 系统需求分析	29
6.2 系统总体架构设计.....	29
6.3 功能模块设计	30
6.4 开发环境与项目结构.....	31
6.5 模型训练与推理模块实现.....	31
6.6 桌面系统实现与测试.....	31
6.7 本章小结	32
第 7 章 总结与展望.....	33
7.1 结论	33
7.2 创新点与局限性.....	33
7.3 后续展望	34
参考文献	35
致谢	37

第 1 章 绪论

1.1 研究背景

疲劳驾驶不是一个消极的精神状态降低造成的，它是一种对于行车安全有威胁的危险行为。驾驶员长时间连续驾驶、夜间驾驶或者睡眠不足的时候，眼部闭合时间就会增多，注意力就会出现间歇性的中断，嘴部打哈欠、头部低垂等现象也会变得越发频繁。当出现驾驶员接收到前面车辆减速、行人突然横穿、车道变道等信号的时候，由于没有得到信息而发生追尾、撞车事故。所以怎样才能高效又精确地对驾驶员的疲劳状况进行检测，这是智能交通、驾驶安全辅助领域所要解决的一个问题。

目前所用的疲劳驾驶检测的方法有生理信号检测法、车辆行为检测法、视觉特征检测法。生理信号方法主要是通过脑电、心电或者眼电等信号来判断疲劳程度，理论依据比较直接，但是需要佩戴传感器，部署成本以及使用舒适性都受到了一定的影响。车辆行为方法是根据方向盘转角、车道偏移、制动行为等信息来推测驾驶状态的一种方法，采集比较容易，但是会受到道路曲率、驾驶习惯、车辆控制策略的影响。相比而言，基于视觉的检测方式可以不接触驾驶员的面部来观察驾驶员的面部状态，在普通摄像头、桌面演示系统上使用较为方便。

闭眼、打哈欠属于比较常见、容易解释的面部表情。闭眼状态下出现的 PERCLOS 等疲劳参数也会产生相应的反映，打哈欠属于困倦、注意力不集中表现的一种形式。传统的视觉方法一般用眼睛的宽高比、嘴部的开合程度或者人工阈值来判断，简单但是对光照、遮挡、姿态的变化比较敏感。随着深度学习的目标检测模型的发展，将闭眼、睁眼、打哈欠等目标检测的任务进行训练，使得模型可以自己学会局部面部特征的学习，而不需要人工阈值的设定。

本文使用的是 YOLO 系列模型来解决疲劳驾驶检测的问题，因为疲劳驾驶检测需要实时、低功耗的部署方式，所以选择 YOLO 系列模型。YOLO 相较于两阶段检测模型而言，把目标定位以及类别预测二者一起放进同一个网络当中来执行，因而它的推理链路比较短，因此它很适合用在图片、视频乃至摄像头帧上实施连续的检测工作。YOLOv8n 参数量小，在普通的计算环境里可以进行演示；闭眼、打哈欠的目标区域小，容易被复杂的背景所削弱，所以本文又加入了 CBAM 注意力机制，在通道、空间两个方向上更加重视眼部、嘴部这些重要区域。根据以上情况，本文就 YOLOv8n 和 CBAM 创建了疲劳驾驶面部识别系统，并且通过实验来检验它的有效性。

1.2 研究意义

本文研究的意义可以从学术价值、应用价值两个方面来理解。

从学术角度来讲，疲劳驾驶的面部识别任务是对于轻度检测，局部小目标特征表现以及状态规则的形成所采取的方式。本文用闭眼、睁眼和打哈欠这三个可以解释的目的来代替整张图像被划分为疲劳或者非疲劳的过程，从而保持模型对于视觉的判断。采用 YOLOv8n 基础模型的基础上加上 CBAM 注意力，在眼部、嘴部等部位的注意效果得到了加强，可作为轻量级的目标检测模型进行进一步的研究参考。

本文除了模型指标之外，还建立了桌面演示系统。系统可以加载训练好的权重，对图片、视频的输入进行检测，得到检测结果即检测框、类别置信度和疲劳状态。这样就使得实验结果可以被直观地观察到，并且方便答辩演示、功能测试以及以后的拓展。虽然本文的系统没有达到真实车载产品的标准，但是可以完整的呈现从数据处理、模型训练、结果分析、软件演示的工程过程，为之后加入摄像头实测、报警策略、时序模型打下基础。

1.3 国内外研究现状

根据检测信号的来源来划分疲劳驾驶检测研究可以分为生理信号法、车辆行为法、视觉特征法这三种。生理信号法是根据脑电、心电、肌电或者眼电等生理信号来判断驾驶员身体状况的，比较容易地体现疲劳的程度。Saleem 等[4]对基于生理信号的驾驶员嗜睡检测方法进行了系统的整理，认为该类方法在理论上是有依据的，但是需要使用传感器佩戴或者专用采集设备，在普通的驾驶环境中部署的成本比较高。Albadawi 等[3]从系统角度对驾驶员嗜睡检测的发展进行了总结，认为视觉、行为、生理信号的方法各有优点，其中非接触式的视觉方法在实际的部署中更加方便。

车辆行为法主要是对车辆的转弯角度、变道行为、制动频率等进行分析。孙超[8]把计算机视觉同驾驶信息结合起来，利用多源信息来加强驾驶安全检测的能力，说明车辆状态信息可以补充驾驶员面部信息。但是车辆行为会受到道路曲率、车速、驾驶习惯、车辆控制系统的影响，疲劳状态和车辆行为之间并不是一一对应的。所以如果只依靠车辆的行为来判断，就容易出现错误的判断。

视觉特征法就是对眼部、嘴部、头部的姿态变化进行研究。罗晓璐[5]用 OpenCV 对驾驶员的疲劳程度进行检测，结构清晰，成本低，但是这种方法一般要手工设置阈值，对于光照、姿态以及个人差别比较敏感。邹晓越[6]对于机器视觉下面部多特征疲劳检测系统做了一些工作，提出使用眼部、嘴部、头部等多特征可以提高判断的可靠性，但是多特征规则的建立会让阈值调节变得更加困难。魏恒建[7]从多通道信息融合的角度来研究疲劳检测系统，重视各个来源的信息互相补充的作用，但是系统的实现难度也比较大。

由于深度学习出现之后,视觉疲劳检测就不再用人的感官感受去做,而用卷积神经网络、目标检测模型等做成了。王丹卿提出用面部特征加上深度学习的方法来实现疲劳检测,说明用深度网络取代人工设计的特征可以减少对人类特征的依赖,在复杂的图像中也可以较好的表达。刘德馨[10]就基于面部特征的疲劳驾驶检测技术进行了研究,从眼部、嘴部局部特征上体现了其解释作用。刘鹏宇[11]把驾驶员面部特征检测同应用系统联系起来,表明面部局部状态可以被当作可以观察到的检测对象。

YOLO 系列速度快,容易使用,但是由于它具有较好的目标检测效果,因此也经常用于实时视觉检测。Redmon 等人[1]提出了 YOLO 目标检测的思想,把目标的定位和分类统一起来建模,为实时检测打下了基础。Ultralytics 的 YOLOv8 不仅有训练、验证、推理所有的接口,并且能够缩减实验复现的成本。丁子原[9]使用了改进后的 YOLOv8 对疲劳驾驶进行轻量级检测,说明 YOLOv8 模型可以用于驾驶员的状态检测。Xie 等人用 YOLOv8 对驾驶员疲劳进行检测,证明了目标检测模型可以被用来解决道路安全问题。周等(2023)用 YOLOv8 做驾驶员面部疲劳检测时发现,使用深度目标检测的方法就可以识别出闭眼、打哈欠这样的一些动作。

注意力机制是提高局部特征表达的一种方式。Woo 等人[2]提出了 CBAM 模块,用通道注意力、空间注意力来重新加权特征图,使得网络更加关注任务相关的区域。疲劳驾驶所造成面部识别出现闭眼,打哈欠这些面的局部特征,是被影响过的图像上的局部现象,很容易受眼镜、灯光等外界因素的影响。在复杂的光照环境之下,刘等利用 YOLOv8 对驾驶员的疲劳情况进行有效的鲁棒性改进实验,可以很好的应对局部有视觉疲劳的检测问题,但是整体上还存在检测困难的现象。所以把注意力机制加到轻量级 YOLO 模型里是有道理的。

综上所述,现有的研究还存在着如下不足之处,第一,传统的视觉方法依靠人工设定阈值,对于复杂的光照以及姿态变化的适应性较差;第二,深度学习的方法虽然检测效果更好,但是在轻量级部署和局部小目标识别之间需要权衡;第三,很多研究只给出了总体指标,没有对 Yawn 等样本较少类别的单独分析;第四,有些研究重模型轻系统,缺少从训练、推理到桌面演示的完整闭环。本文以 YOLOv8n 为基准,在 PyQt5 的 GUI 下实现了一款桌面式闭眼、睁眼、打哈欠的检测系统。

就研究路线而言,并非一味地重视各个指标的相加,而应该从现实的疲劳驾驶识别链路上来考虑。首先要得到可以体现眼部和嘴部状态的数据。其次要选择兼顾速度和精度的检测模型。最后需要将模型的输出转化为系统可以理解的疲劳状态。这样一条路线可以把理论模型、实验验证、软件实现三者联系起来,并且可以解决目前研究中重算法轻落地的问题。

1.4 本文主要研究内容

本文主要完成以下工作:

(1) 数据集的创建。本文使用公开整理的 Driver_Drowsiness_YOLO 数据集，对 YOLO 格式标注进行解析、训练集、验证集、测试集划分检查、类别含义说明以及类别分布统计。对于 Eyeclosed、Eyeopen 和 Yawn 三种类型的样本，分析样本不平衡以及场景覆盖率不足会对之后的模型训练造成怎样的影响。

(2) 基线模型的训练。本文用 YOLOv8n 做基线模型，采用相同的输入尺寸、训练轮数、batch size、优化器、学习率策略进行闭眼、睁眼、打哈欠三种目标检测训练，并得到 Precision、Recall、mAP50、mAP50-95、PR 曲线、F1 曲线以及混淆矩阵等结果。

(3) 模型改良。为了解决由于面部的目标区域较小、眼睛和嘴巴容易受到光线或者遮挡而造成困难的问题，对 YOLOv8n 的 Backbone 末端、Neck 输入端进行 CBAM 注意力模块的添加，得到 YOLOv8n+CBAM 改进模型，与原始基线模型在相同的数据集上进行对比。

(4) 实验对比。本文以 CBAM 整体指标、类别指标、PR 曲线、F1 曲线、混淆矩阵等作为主要分析对象，主要探究 CBAM 对 Recall、Yawn 类别的检测效果影响，解释出现的 Precision 有微降、Yawn 样本少但是召回率高的实验现象。

(5) 系统实现。本文以 Python、PyTorch、OpenCV、PyQt5 为开发工具，实现了桌面演示系统，完成了模型加载、图片或者视频输入、检测结果绘制、疲劳状态判断、截图保存、日志记录等模块的功能，使得模型的结果可以通过软件界面直观地展现出来。

本文的主要贡献是建立了一套面向闭眼、睁眼和打哈欠三种可以解释的目标的疲劳驾驶面部识别实验流程，第二，在轻量级 YOLOv8n 模型中加入了 CBAM 注意力模块，用同条件对比的方式证明它对于局部疲劳特征的检测效果，第三，将模型训练、实验分析、PyQt5 桌面系统整合在一起，形成一个可以复现、可以演示的毕业设计完整闭环。

1.5 论文组织结构

本文共七章。第一章介绍研究背景、研究意义、国内外研究现状以及本文主要研究内容。第二章概述了相关的技术理论基础，说明了 YOLOv8、CBAM、深度学习框架、图像处理、桌面界面技术的基本概念以及选用的依据。第三章是对数据集的构建以及预处理进行介绍，包括数据来源、类别划分、标注格式、数据划分、增强策略。第四章为模型与算法设计，基线模型、CBAM 嵌入方案、疲劳状态判断方法、训练策略等都在本章节中展开。第五章分别用对比实验、曲线图、混淆矩阵来评价改进方案的效果。第六章主要介绍系统需求、总体架构以及各个模块的实现。第七章总结全文工作，提出创新点、局限性以及今后的研究方向。

第 2 章 相关技术理论基础

第一章已经明确本文研究的是疲劳驾驶面部识别的任务，后面模型的训练以及系统的实现都需要依靠目标检测、注意力机制、深度学习训练框架、图像处理、桌面界面开发等基本技术。本章不是本文系统设计的细节，而是对这些技术的主要原理及选择的原因进行说明。与后面章节的关系是，本章解释技术基础，第四章说明具体的模型配置和算法规则，第六章说明系统的实现方式。

2.1 YOLOv8 目标检测算法

YOLO 系列算法属于一阶段目标检测方法，其主要思想就是把目标的定位以及类别预测都整合到同一个网络当中来实现。相比先生成候选区域再进行筛选的方式来说，YOLO 没有候选框筛选的过程，在推理上更加高效，更加适合实时检测，可以用于图片、视频帧、摄像头帧等场景。疲劳驾驶面部识别系统对于连续的输入数据，要快速得到闭眼、睁眼、打哈欠的结果，这就需要有较强的实时性。因此使用 YOLO 这类模型较为合适。

YOLOv8 通常由 Backbone、Neck 和 Head 三部分构成。Backbone 提取图像的多层特征，Neck 把不同尺度的信息合并起来，Head 做边界框的回归，之后做类别预测[13]。就本文的任务来说，眼睛和嘴巴在图像中所占的比例小，模型要保持局部纹理的同时还要利用高层语义信息。YOLOv8 对于不同大小的小区域的面部目标检测有铺垫的作用。

本文选择 YOLOv8n 而不是更大的模型，主要是因为实验条件以及系统演示的需求所决定的。YOLOv8n 参数量少，训练、推理成本低，在服务器上完成模型训练之后，可以将模型迁移到本地的 Windows 环境进行演示。本文在第四章只对 YOLOv8n 进行了配置与改进，不再对 YOLOv8 的原理进行介绍，而是在 CBAM 嵌入位置、输入尺寸、训练参数方面做了说明。

2.2 CBAM 注意力机制

CBAM (Convolutional Block Attention Module) 是一种轻量级卷积注意力模块，由通道注意力和空间注意力串联组成[2]。通道注意力回答的是“哪些特征通道更重要”，空间注意力回答的是“图像中哪些位置更重要”。通过再加权来对特征图进行处理，使网络更加重视和任务相关联的局部区域。

CBAM 的通道注意力一般用全局平均池化、最大池化得到通道描述，然后用共享网络学习通道权重；空间注意力是在通道维度上压缩特征，用卷积生成空间权重图。其基本加权过程如公式 (2-1) 所示： $F' = Mc(F) \times F$ ， $F'' = Ms(F') \times F'$ (2-1)。其中 F 是输入特征， Mc 是通道注

意力权重， M_s 是空间注意力权重[2]。

CBAM 适合本文的任务是因为闭眼和打哈欠都是局部面部区域的变化，目标区域小并且容易受到光照、眼镜、姿态、遮挡的影响。只有普通的卷积提取出全局特征的时候，眼部、嘴部等区域就会被背景或者人脸的其他部分所弱化。CBAM 可以不明显增加参数量就加强关键通道、关键空间位置，所以适合嵌入到轻量级 YOLOv8n 模型中。

2.3 PyTorch 与 Ultralytics 框架

PyTorch 是常用的深度学习框架，有动态图机制、接口灵活、调试方便等特点，适合做模型的训练、验证、推理等工作[17]。Ultralytics YOLOv8 基于 PyTorch 实现，提供数据读取、模型构建、训练验证、指标评估和权重保存等接口[13]。这些接口可以减少由于重复工程造成成本的增加，将本文的研究重点放在数据分析、模型改进以及实验上三个方面。

本文后面训练基线模型和改进模型时，都用 PyTorch 和 Ultralytics 框架来完成。因此保证两组模型的数据划分、输入尺寸、优化器、训练轮数、评价过程等各方面的相同，从而减少由于训练流程不同造成的实验误差。

2.4 OpenCV 图像处理技术

OpenCV 是一个常用的计算机视觉库，可以实现图像的读取、视频的解码、摄像头的采集、尺寸的缩放、颜色的转换、检测框的绘制以及结果的保存等功能[18]。疲劳驾驶检测系统需应对图片、视频、摄像头等多种输入方式，还要把模型所产的边界框、类别名称、置信度、状态提示这些结果画成图表，因此 OpenCV 成了模型推理以及显示的结果之间的桥梁。

本文系统中 OpenCV 的主要作用是读取输入帧、对图像进行预处理、检测结果的绘制以及截图保存。模型本身产生类别、边界框，OpenCV 把它们变成用户能看见的图像画面。

2.5 PyQt5 桌面界面技术

PyQt5 是 Python 环境下常用的跨平台桌面界面开发工具，可以创建窗口、按钮、标签、文件选择、线程、信号槽等组件[19]。对于毕业设计系统来说，仅仅有模型指标是不能体现整个工作过程的。系统还要具有输入选择、检测启动、状态显示、日志记录、截图保存等功能。

本文选择 PyQt5 是因为它可以与 Python 模型推理代码、OpenCV 图像处理代码直接连接。使用 PyQt5 线程和信号槽的方式，把视频检测的任务放在耗时比较多的推理过程之外，从而减少界面出现卡顿的可能性。

2.6 模型运行与可视化基础

本文的重点在于训练权重的加载、推理、展示在本地桌面环境中进行，不会涉及到跨平台部署或者模型格式的转换。因此本节对模型的运行情况与结果进行论述。模型运行过程主要是权重文件的读取，输入图像的预处理，模型的前向推理，检测结果的解析以及结果的可视化。

因此，本章只概述后续设计所需的关键技术。YOLOv8 用于实时定位疲劳相关目标，CBAM 用于增强眼部和嘴部局部特征，PyTorch、Ultralytics、OpenCV 和 PyQt5 分别支撑训练、推理、图像处理和界面展示。

就工程而言，模型的运行稳定性同权重路径、类别名称、输入尺寸以及推理设备是否一致有关。本文之后系统的实现保持了模型权重的选择、输入路径的选择和日志提示的功能，在不同实验文件之间进行切换时，更容易找到问题。

2.7 本章小结

本章概述了 YOLOv8、CBAM、PyTorch、Ultralytics、OpenCV 和 PyQt5 等基础技术，明确了它们在检测、改进、训练和展示中的作用，为后续模型设计、实验分析和系统实现提供支撑。

第 3 章 数据集构建与预处理

在明确相关技术后，模型训练还需要规范的数据支撑。本章围绕疲劳驾驶面部数据集，说明数据来源、标注类别、划分方式、类别分布、输入预处理和数据局限性。

3.1 数据来源与采集场景说明

本文使用 Roboflow 平台整理的 Driver_Drowsiness_YOLO 数据集作为主要训练数据。此数据集属于驾驶员面部状态检测任务，图像内容以人脸局部为主，含有闭眼、睁眼、打哈欠等与疲劳驾驶有关的面部表情。数据集已经整理成 YOLOv8 可以读取的目录结构，分为 images 和 labels 两种文件，标注文件用归一化边界框格式。

本文共用到的图像有 5413 张，其中训练集为 3790 张，验证集为 542 张，测试集为 1081 张。图像格式主要是常见的 RGB 图像文件，分辨率在原始数据中并不完全相同，训练前都经过了缩放和填充处理，得到 640×640 的输入。由于数据来源于公开平台整理，而不是本文所使用的自行车载采集，所以样本场景主要是人脸正面或者近正面的状态，对于真实的车辆振动、夜间弱光、强光反射、复杂的背景、大角度侧脸等的覆盖还比较有限。

本文并没有直接将整幅图像分为疲劳或者非疲劳，而是先检测出可以解释的面部局部目标，再把检测结果交给疲劳状态判定规则。这样可以保持检测框、类别置信度以及状态的源头，从而使得系统所输出的内容更为清晰地被人们所看到，并且也可以进行进一步的解释。本文的数据集类别以及标注含义如表 3-1 所示。

表 3-1 数据集类别与标注含义

类别编号	类别名称	中文含义	检测含义
0	Eyeclosed	闭眼	眼睛闭合或接近闭合，是疲劳风险的重要表征
1	Eyeopen	睁眼	眼睛处于正常睁开状态，可作为非疲劳参考特征
2	Yawn	打哈欠	嘴部明显张开，反映疲劳或困倦趋势

3.2 标注格式与数据集划分

YOLO 标注文件是以 txt 文本的形式保存的，每一行代表一个目标框。每行格式为 class_id、x_center、y_center、width、height。class_id 表示类别号，后四个数值分别为边界框的中心坐

标、宽度和高度。坐标均是相对于图像宽度和高度归一化的。归一化标注可以减小由于原始分辨率不同而产生的影响，在训练的时候也可以用缩放、填充和批量增强来处理。

本文所用数据集有 5413 张图片，已经分成训练集、验证集和测试集。训练集用来学习模型参数，验证集用作训练过程中性能的评价和模型的选择，测试集用在后续的系统演示和泛化效果的观察上。具体的划分如表 3-2 所示。

表 3-2 数据集划分统计

数据划分	图片数量	标签文件数量	用途
训练集	3790	3790	用于模型参数学习和数据增强
验证集	542	542	用于训练过程评估和模型选择
测试集	1081	1081	用于系统演示和检测效果观察
合计	5413	5413	覆盖三类疲劳相关面部特征

3.3 类别分布统计与样本不均衡分析

为了更好的了解数据集的质量，本文对 train、valid、test 这三个划分的标签文件进行统计，得到三种目标实例的数量。类别分布统计结果如图 3-1 所示。

疲劳驾驶数据集类别分布统计

统计范围：train、valid、test 三个划分的 YOLO 标签文件

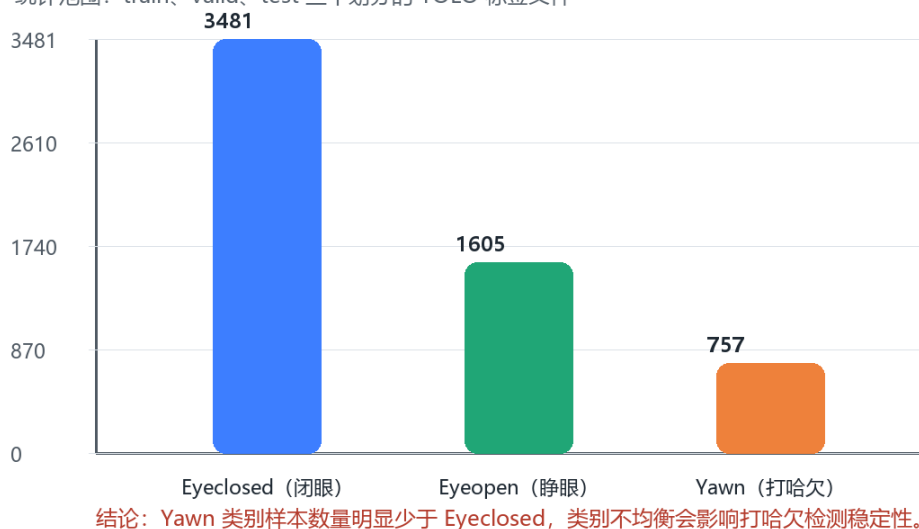


图 3-1 数据集类别分布统计

从类别分布可以看出，Eyeclosed 类别实例数量为 3481，Eyeopen 类别为 1605，Yawn 类

别为 757。Eyeclosed 和 Yawn 数量之比大约为 4.6 比 1，打哈欠的样本远远少于闭眼的样本。该比例并不会使模型完全失效，但是对于少数类的学得会存在很大的困难，尤其是在嘴部张开幅度、脸部角度、遮挡变化等都属于 Yawn 类别的情况下，很容易出现误检或者漏检。

本文没有使用额外的采样、类别重加权或者专门的加权损失函数来处理类别不平衡问题，只是使用了 Ultralytics 默认的训练过程。因此第五章除了总体 mAP 之外，还会单独比较各个类别 Precision、Recall 和 mAP 指标，主要研究 Yawn 类别的样本少时的表现。

3.4 数据预处理流程

本节主要对输入预处理过程进行说明，训练增强策略在第四章训练参数设置中统一说明。由于原始图像的分辨率、宽高比等信息并不完全相同，所以直接缩放到一个固定的尺寸会造成面部局部的目标形变。因此本文使用 YOLOv8 常用的 letterbox 方法进行处理，即先按原图宽高比例等比例缩放，再在空白处填充，得到最终输入的尺寸为 640×640。

在像素处理上，输入图像会把整数像素值转成模型可以处理的浮点张量，再把像素范围归一化到 0 到 1 之间。归一化可以减少由于图像的亮度、像素大小等不同而给训练带来的影响。对于边界框，YOLO 标注中的 `x_center`、`y_center`、`width` 和 `height` 均以图像宽高为基准归一化保存。经过缩放和填充之后，程序会根据缩放比例和填充偏移同步变换边界框坐标，使得目标框仍然对应到正确的头部部位。

以上述预处理方式对于疲劳驾驶面部识别起到了重要的作用。闭眼、嘴部张开都是局部小目标，缩放或者填充之后标签坐标不匹配，模型就会学到错误的位置，从而影响后面检测的结果。因此，在训练和推理阶段使用相同的输入尺寸和预处理方式来保证模型的训练、验证以及系统的演示所用的数据形式是相同的。

3.5 数据集不足与影响分析

虽然上述数据集可以用于本文闭眼、睁眼、打哈欠三种目标的检测实验，但是仍然存在一些不足。

(1) 覆盖面窄。主要是从公开的资料中整理得出结论的，很少有真实的车载摄像头样本。车辆行驶过程中受到的振动、摄像头安装角度的变化、驾驶室背景干扰等没有被完全考虑进去，所以模型在真实道路场景中泛化的程度还需要进一步的检验。

(2) 样本种类单一。夜间弱光、强光反光、戴眼镜、戴口罩遮挡、侧脸、低头等样本数量很少。这些都会影响到眼部以及嘴部的可见程度，而且还会加大 Eyeclosed、Eyeopen 和 Yawn 三者之间的混淆几率。

(3) 没有时序标注。本文的数据集主要是单帧图像，没有连续视频序列中闭眼的持续时

间、眨眼的频率、打哈欠的频次的标注。所以本研究只能依靠单帧检测的结果以及滑动窗口的方式，不能模拟出由于疲劳而产生的累积过程。

就实验影响而言，数据集的大小及分布情况都会对模型评价结果造成影响。Eyeclosed 样本数最多，模型可以学到较为稳定的闭眼特征，Yawn 样本数最少，模型对这些样本的评价会受到个别样本、场景的影响较大。本文在第 5 章中专门对 Yawn 类别指标进行分析，目的是为了避开用总体 mAP 来掩盖少数类别的不稳定情况。如果以后要增加数据集的话，应该先增加打哈欠、侧脸、遮挡、夜间等样本，使训练集更接近真实的驾驶环境。

以上不足会使得模型在真实的驾驶环境中不稳定，这也是第七章后面提到的增加数据集、加入时序模型的原因。但是就毕业设计目标来说，本文所用的数据集已经包含了三个重要的疲劳面部目标，并且具有规范的 YOLO 标注以及明确的训练、验证、测试划分，可以满足本文进行模型训练、对比实验以及桌面系统的演示。

3.6 本章小结

本章说明了数据来源、标注格式、数据划分、类别分布、预处理流程和数据不足。通过这些内容，可以明确实验数据的组成与限制，为后续模型训练和结果分析提供依据。

第 4 章 模型/算法设计与改进

在完成数据集说明后，本章进入模型与算法设计，重点介绍检测任务、YOLOv8n 配置、CBAM 嵌入位置、疲劳判定规则和训练参数设置。

4.1 疲劳面部特征检测任务分析

本文检测目标包括 Eyeclosed、Eyeopen 和 Yawn 三类。三类目标并不是随意选出来的面部类别，而是和疲劳驾驶的判断有关联、可以解释的视觉线索。Eyeclosed 表示眼睛闭合或者接近闭合的状态，可以用来近似地表示 PERCLOS 中“眼睛闭合时间比例”的视觉来源；Eyeopen 给出正常睁眼的参照，有利于区分闭眼和非疲劳状态；Yawn 代表嘴部明显的张开，可以体现困倦和注意下降的趋势。

相对于车辆、行人这些大目标而言，眼部和嘴部在整张图像里所占的比例较小，边界形态很容易受到头部的姿态、拍摄的距离、光照情况、戴眼镜的情况以及被遮挡的影响。特别是 Yawn 类别，嘴部张开的程度是连续的，普通说话、表情变化、打哈欠之间有视觉上的相似性。因此模型不但要具有实时检测的能力，而且要在局部区域里提取出能区别的特征。

本文使用的是“目标检测加规则判定”的两阶段方法。第一部分利用 YOLOv8n 或者 YOLOv8n+CBAM 检测到的面部位置、置信度来判定疲劳程度，第二步按照连续帧或者滑动窗口中有闭眼和打哈欠的情况来判定疲劳等级。相对于直接输出的疲劳二分类而言，它有更易于理解的结构，在给出疲劳判断原因的同时还可以输出具体的检测框和类别置信度，使得用户能够清楚地知道疲劳是由闭眼还是打哈欠引起的。

4.2 YOLOv8n 基线模型设计

基线模型用的是 YOLOv8n。本节主要介绍本文是如何配置、使用 YOLOv8 模型的。YOLOv8n 是 YOLOv8 系列中参数量最小的一个版本，在有限的训练资源以及普通本地演示环境下可以使用。本文使用该模型作为比较基准，能比较清楚地看出 CBAM 注意力模块带来哪些性能上的变化。

基线训练的输入尺寸设为 640×640 ，这是因为 640×640 是 Ultralytics YOLO 训练中常用的输入尺寸，在检测精度和显存使用上比较合适。训练轮数设置为 50 轮，由于本文的数据集规模中等，如果轮数太少就会导致收敛不足，如果轮数太多，在毕业设计实验条件下又会增加训练成本。batch size 设为 16，以适配 Tesla V100 32GB 显存并保证训练稳定性。

优化器用 AdamW，学习率用 Cosine LR 策略。AdamW 可以加入到梯度更新中去进行权重衰减，从而降低过拟合风险，在训练的后期使用余弦退火式的学习率衰减来使得网络在后

期更加平稳。基线模型训练得到的权重文件、结果表、PR 曲线、F1 曲线、混淆矩阵等文件作为第五章与改进模型比较的依据。

4.3 CBAM 注意力模块嵌入方案

为了提高模型对于局部疲劳特征的代表能力，在 YOLOv8n 的 Backbone 末端和 Neck 输入端分别加入了 CBAM 模块，也就是在 SPPF 模块的输出之后加上注意力加权。该位置处在高层语义特征输出之后、特征融合之前，在进入 Neck 多尺度融合之前就对重要的通道以及空间区域加以强调。相比于在浅层加入注意力，该位置的语义更加强大；相比于在检测头之后加入注意力，该位置又会影响到后续特征融合的过程。

第 2 章已经介绍了 CBAM 的基本原理，本节不再重复其通道注意力和空间注意力的计算过程。本文具体实现时，在模型配置文件 `configs/yolov8n_cbam.yaml` 中注册 CBAM 模块，并将其插入 YOLOv8n 主干网络 SPPF 输出之后。训练入口读取该配置文件之后，由 Ultralytics 框架根据配置来建立网络结构。为了保证训练的稳定性，模型仍然从 YOLOv8n 预训练权重中初始化，并且对 CBAM 部分采用默认初始化的方式进行学习。

改进的方式有两个特点。第一，修改范围小，没有改变检测头的输出类别和边界框的方式，所以可以和原来的 YOLOv8n 训练、验证和推理流程兼容。第二，参数增加不多，改进模型的参数量约为 3.08M，比基线模型约 3.01M 只增加了 0.07M，仍然满足轻量化演示的要求。第五章用同条件对比实验来检验该嵌入方案是否能提高检测性能。

4.4 疲劳状态判定算法

YOLOv8 检测模型会对于每帧输出目标的类别、置信度以及边界框。把检测结果转化为疲劳程度的时候采用滑动窗口统计的方式。窗口长度取 N 帧，本文中 N 取 30，在窗口内闭眼帧数记为 C_{eye} ，打哈欠帧数记为 C_{yawn} ，则闭眼比例 $P_{eye}=C_{eye}/N$ ，打哈欠次数为 C_{yawn} 。只有当目标置信度大于等于 0.25 的时候，这个帧的检测结果才会被加入到状态统计当中。

本文把疲劳状态分成正常、疑似疲劳、疲劳三类。若 $P_{eye}<0.30$ 且 $C_{yawn}=0$ ，则系统输出正常；若 $0.30\leq P_{eye}<0.50$ ，或窗口内出现 1 次打哈欠，则输出疑似疲劳；若 $P_{eye}\geq 0.50$ ，或连续两个滑动窗口均出现打哈欠，或单个窗口内 $C_{yawn}\geq 2$ ，则输出疲劳。以上阈值不是医学诊断标准，是根据毕业设计演示需要以及可解释性而设定的经验阈值，之后会用真实的驾驶视频进行校准。

系统在程序实现的时候，会维持一个固定长度的队列来保存最近 N 帧有效的检测结果。每读入一帧图像之后，先通过置信度阈值来过滤掉那些低可信的检测框，然后再更新闭眼帧

数以及打哈欠次数。接着根据表 4-1 中给出的规则进行当前状态的计算。当风险等级高的结果经常变化的时候，界面端就会先显示风险等级较高的结果，防止出现漏判或者误判的情况发生。该策略比较简单，可以用来在桌面演示系统中解释和复现。

规则的优点是简单、易懂，可以将模型检测的结果转化为界面可以显示的状态文本。它依然依靠窗口长度、阈值来建模眨眼频率、闭眼的时间、驾驶员个人的差异。因此，在第七章中本文将时序模型以及真实视频数据作为之后改进的方向。疲劳状态判断的规则表，如表 4-1 所示。

表 4-1 疲劳状态判定规则

状态	量化判定依据	系统显示含义
正常	窗口长度 $N=30$ 帧；闭眼比例 $P_{eye}<0.30$ ，且打哈欠次数 $C_{yawn}=0$	驾驶员面部状态相对正常
疑似疲劳	$0.30 \leq P_{eye} < 0.50$ ，或窗口内检测到 1 次打哈欠	提示用户关注当前状态
疲劳	$P_{eye} \geq 0.50$ ，或连续两个窗口出现打哈欠，或单窗口 $C_{yawn} \geq 2$	提示存在明显疲劳风险

4.5 训练策略与参数设置

为了保证两组对比实验的公平性，YOLOv8n 基线模型和 YOLOv8n+CBAM 改进模型使用相同的划分数据集、输入尺寸、训练轮数、batch size、优化器、学习率策略、数据增强设置。第 3 章已经说明输入缩放、填充和归一化预处理，本节只说明训练阶段使用的增强和参数。

在训练过程中使用了 Mosaic、MixUp、HSV 颜色变化。Mosaic 是将多幅图像融合在一起构成一个训练样例，从而加大了目标尺度与背景组合变化的程度，而 MixUp 则采用图像混合的方法来削弱模型对于单个图像纹理的过分依赖，同时 HSV 色彩扰动使得画面的亮度、色彩出现不同程度的波动，进而给模型带来不一样的光照环境下物体的形态。由于本文的数据集中真实的驾驶场景较少，所以需要对其进行一定的增强以提高模型的泛化性。

本文训练在 Slurm GPU 节点上完成，GPU 为 Tesla V100 32GB，batch size 为 16，输入尺寸为 640，训练轮数为 50。优化器采用 AdamW，初始学习率设为 0.001，权重衰减设为 0.0005，学习率策略为 Cosine LR，使用 YOLOv8n 预训练权重初始化，不额外使用标签平滑。验证阶段记录 Precision、Recall、mAP50、mAP50-95、类别指标、PR 曲线、F1 曲线和混淆矩阵。模型训练参数设置如表 4-2 所示。

表 4-2 模型训练参数设置

参数	取值
基础模型	YOLOv8n
改进模型	YOLOv8n + CBAM
输入尺寸	640×640
训练轮数	50
Batch size	16
优化器	AdamW
初始学习率	0.001
权重衰减	0.0005
学习率策略	Cosine LR
预训练权重	使用 YOLOv8n 预训练权重初始化
数据增强	Mosaic、MixUp、HSV 色彩扰动
标签平滑	未额外启用
验证指标	Precision、Recall、mAP50、mAP50-95、PR 曲线、F1 曲线、混淆矩阵

4.6 本章小结

本章完成了疲劳面部特征分析、YOLOv8n 基线配置、CBAM 嵌入方案、疲劳判定规则和训练参数设置，形成了可对比的基线模型与改进模型，为第五章实验分析奠定基础。

第 5 章 实验设计与结果分析

在模型方案确定后，本章通过实验环境、评价指标、对比结果、曲线和混淆矩阵，分析 CBAM 对疲劳驾驶面部识别效果的影响。

5.1 实验环境与评价指标

本文实验分为服务器训练和本地演示两部分。模型训练在 Slurm GPU 节点上完成，GPU 为 Tesla V100 PCIE 32GB，Python 版本为 3.11.11，PyTorch 版本为 2.5.1，Ultralytics 版本为 8.4.45。本地演示在 Windows CPU 环境中完成，主要用于验证权重加载、图片/视频输入、检测结果绘制和疲劳状态显示功能。

评价指标包括 Precision、Recall、mAP50、mAP50-95 和推理耗时。Precision 是指预测结果中正确的目标所占的比例，Recall 是指真实的目标被检测出来的比例。疲劳驾驶的检测主要是从风险状态是否被忽略的角度来分析，具有很高的分析价值。mAP50 是 IoU 阈值为 0.5 时的平均精度，mAP50-95 是综合多个 IoU 阈值得到的平均精度，更能反映定位的质量。表 5-1 中推理耗时是在同一验证流程下统计得到的，输入尺寸为 640×640，batch size 为 1，使用 FP32 推理，只用来比较两组模型相对耗时的差异。

5.2 两组对比实验设计

本文设置两个对比实验，第一组是 YOLOv8n 基线模型，第二组是 YOLOv8n+CBAM 改进模型。两组实验所用的训练集、验证集以及测试集是相同的，输入尺寸、训练轮数、batch size、优化器以及增强策略也都是相同的。唯一的结构不同就是改进模型在 Backbone 末端和 Neck 输入端加了 CBAM 注意力模块。因此这是简化版的结构对照实验，用来研究 CBAM 对检测结果的影响。

对比实验主要回答三个问题，第一，CBAM 加入之后整体 mAP、Recall 是否提高；第二，注意力机制能否改善样本少、局部变化大的 Yawn 类别的表现；第三，精度的提升是以明显的推理耗时增加为代价吗。为了防止只看一个指标而造成片面的判断，本文用整体指标、类别指标、PR 曲线、F1 曲线、混淆矩阵综合分析。基线模型和 CBAM 改进模型的对比结果见表 5-1。

表 5-1 基线模型与 CBAM 改进模型对比结果

模型	Precision	Recall	mAP50	mAP50-95	参数量	推理耗时
YOLOv8n	0.6868	0.9114	0.8347	0.5368	约	0.9 ms

					3.01M	
YOLOv8n +	0.6840	0.9370	0.8580	0.5470	约	1.1 ms
CBAM					3.08M	

5.3 PR 曲线与 F1 曲线分析

PR 曲线是用不同置信度的阈值来反映 Precision 和 Recall 的关系。基线模型和 CBAM 模型的 PR 曲线分别见图 5-1 和图 5-2。曲线向右上方方向偏移的时候，在高的召回率下也可以得到高的准确率。从曲线可以发现 CBAM 模型的 PR 曲线更接近于右上部分，即该模型对于重要的局部区域有较大的反应。

从类别上讲，Eyeclosed、Eyeopen 的目标主要在眼部位置，边界比较稳定，所以这两类 PR 曲线表现较好。Yawn 类别的变化受嘴部张开程度、脸部角度以及表情的改变影响更大，曲线的起伏也更为明显。CBAM 用通道、空间再加权来加强嘴部区域特征，使得 Yawn 类别的误检增幅小很多，和表 5-1 中 Recall 提高的现象相吻合。

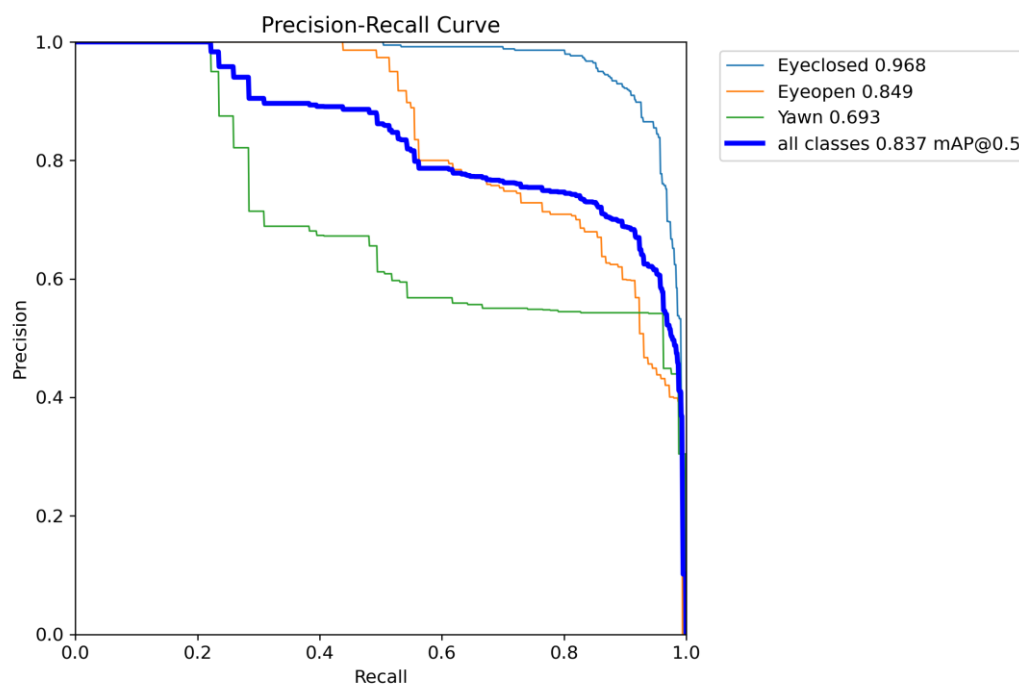


图 5-1 YOLOv8n 基线模型 PR 曲线

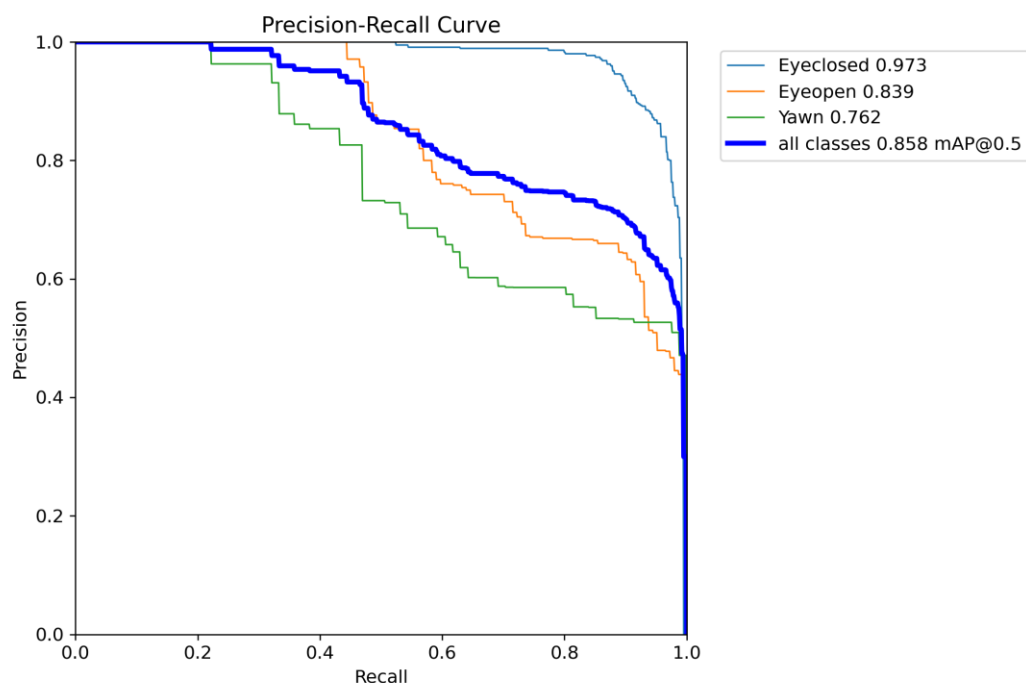


图 5-2 YOLOv8n + CBAM 模型 PR 曲线

F1 值是用 Precision 和 Recall 来评价各个置信度阈值下模型整体检测的平衡。基线模型和 CBAM 模型的 F1 曲线分别见图 5-3 和图 5-4。CBAM 模型在较宽的阈值范围内有较好的 F1 表现，说明它不是在某个特定的阈值上偶然提高的，在整个置信度的变化过程中有更好的综合表现。

图 5-1 到图 5-4 是训练验证过程中自动导出的曲线图，由于原始导出分辨率的影响，局部的文字在论文中显示不清楚。本文保留这些图来展示实验趋势，以后如果要正式归档的话，可以从训练结果 CSV 或者原始绘图脚本中导出更高的分辨率图片，从而提高论文的图像质量。

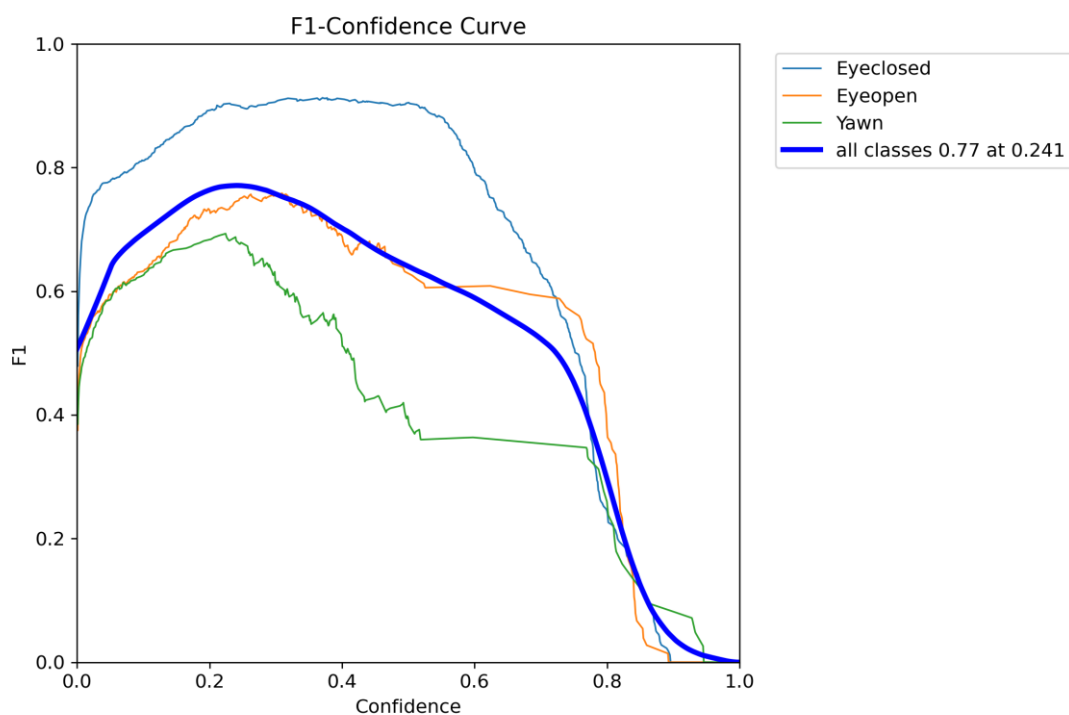


图 5-3 YOLOv8n 基线模型 F1 曲线

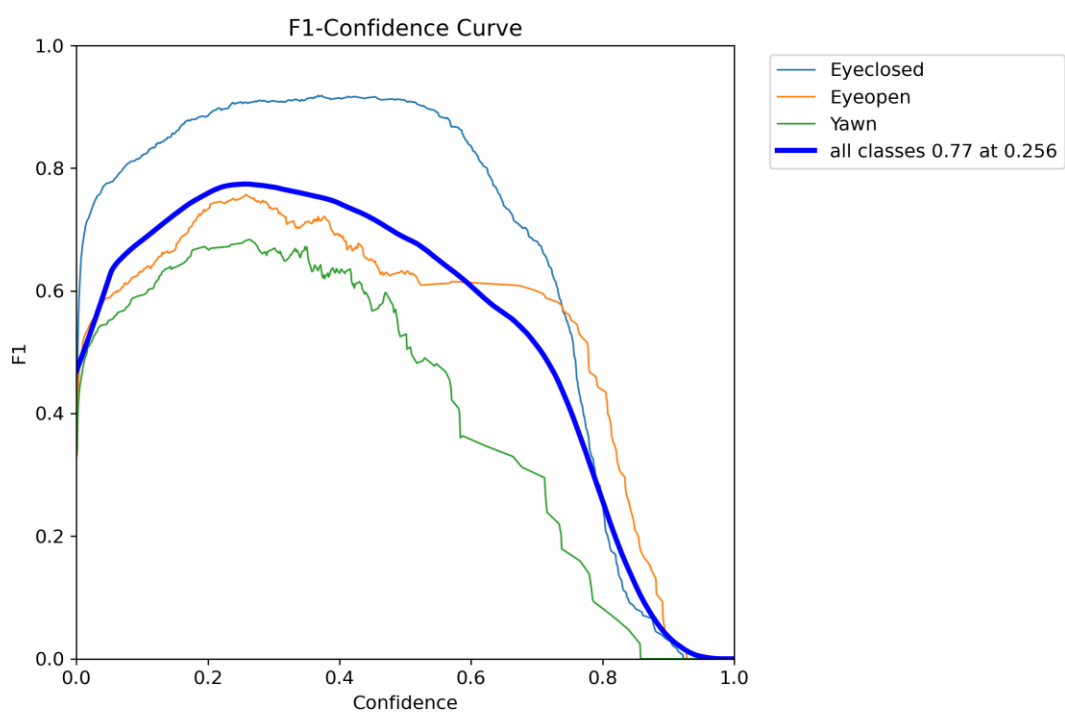


图 5-4 YOLOv8n + CBAM 模型 F1 曲线

5.4 类别结果与混淆矩阵分析

表 5-2 CBAM 模型各类别检测结果

类别	Images	Instances	Precision	Recall	mAP50	mAP50-95
Eyeclosed	278	353	0.871	0.946	0.973	0.574
Eyeopen	107	144	0.657	0.889	0.839	0.531
Yawn	81	81	0.524	0.975	0.762	0.535

从表 5-2 可以看出, Eyeclosed 类别 mAP50 达到 0.973, 检测效果最好; Eyeopen 类别 mAP50 为 0.839, 表现相对稳定; Yawn 类别 mAP50 为 0.762, 低于眼部类别, 但相比基线模型约 0.693 已有明显提高。Yawn 类别样本数量只有 757 个, 比 Eyeclosed、Eyeopen 少, 但是 Recall 高, 可能是由于测试集中打哈欠样本形态比较集中造成的, 当嘴部张开幅度明显的时候, 模型更容易召回该目标, 同时 Precision 低也说明模型会把普通张嘴、说话或者局部遮挡误判为 Yawn。因此高 Recall 并不意味着 Yawn 类别的数据是可以信赖的, 应该同时参考 Precision 和混淆矩阵一起考虑。

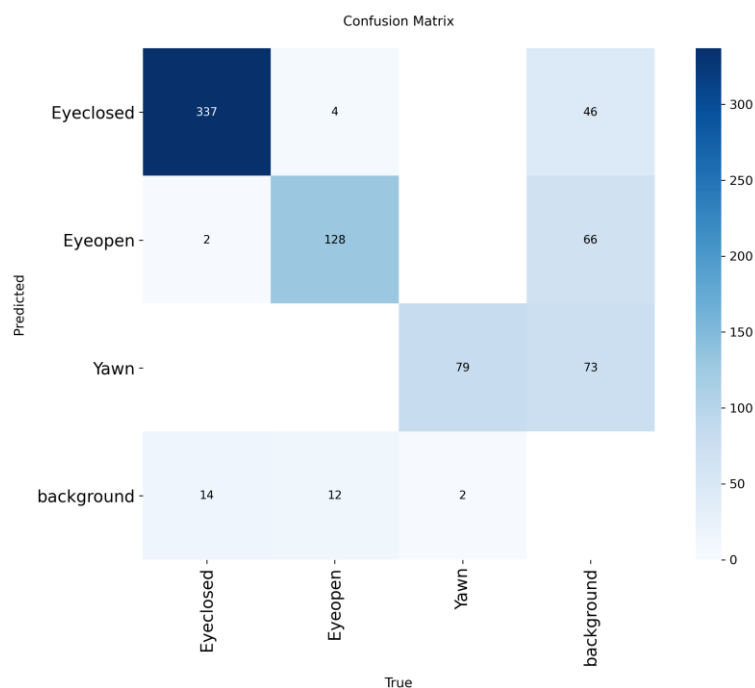


图 5-5 YOLOv8n + CBAM 模型混淆矩阵

混淆矩阵能很好地体现各个类别误判、漏判的关系, CBAM 模型的混淆矩阵如图 5-5 所

示。Eyeclosed、Eyeopen 都是眼部附近的物体，低分辨率、眼镜遮挡或者眼睛半闭的时候容易造成它们的混淆。Yawn 类别主要是由嘴部张开的幅度、面部的角度以及遮挡所决定的。从混淆矩阵可以看出，后面如果想提高模型的性能，就要增加打哈欠、复杂姿态样本。

需要注意的是，Yawn 类别召回率高并不能说明该类别已经稳定了。由于测试集中打哈欠样本数量较少，少量样本正确的检会出现明显的提高 Recall。打哈欠一般会有一个较大的嘴部开合范围，目标外观比半闭眼更加明显，所以模型更容易被召回。但是普通的张嘴、说话或者头部的转动都会使模型把它们当作 Yawn，从而导致 Precision 下降。该现象同第 3 章数据不均衡分析的结果是一致的。

5.5 系统演示结果分析

系统演示结果如图 5-6、图 5-7 和图 5-8 所示。三组结果分别对应的是单张图片的检测结果、桌面界面打哈欠检测的结果以及完整的检测结果展示，用以证明模型权重是否可以从训练环境迁移到本地的 Windows 演示环境中。

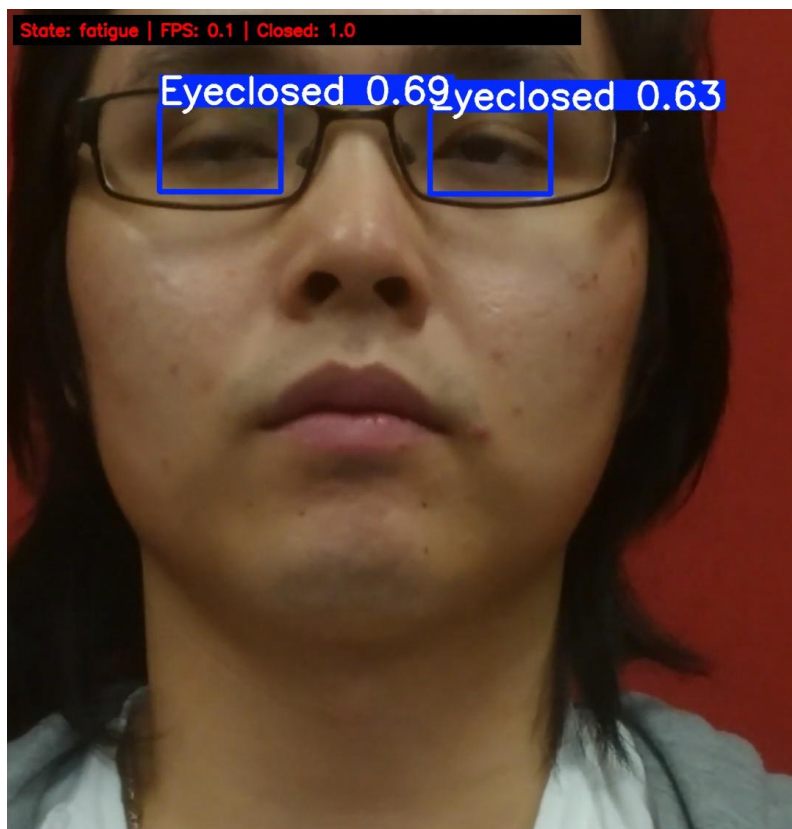


图 5-6 单张图片检测结果示例

图 5-6 是对面部图像闭眼位置的定位，在检测框上标注了类别名称和置信度。由此可以知道模型的输出包含疲劳的状态，也包含了具体的视觉证据。对于毕业设计演示来说，用可视化的结果比单纯的显示“疲劳/正常”更有利于说明算法的判断依据。



图 5-7 桌面系统打哈欠检测界面

图 5-7 中桌面系统可以加载模型并显示打哈欠检测结果。当检测到 Yawn 目标的时候，系统就会把该帧的结果送到疲劳判定模块。滑动窗口内打哈欠次数大于表 4-1 中给出的阈值的时候，界面就会由正常状态转为疑似疲劳或者疲劳。由此可以看出，本章所设计的规则可以和模型的输出相衔接，而不是停留在理论的描述上。



图 5-8 桌面系统检测结果示例

图 5-8 为系统检测结果、疲劳状态、截图保存路径以及日志路径等信息的展示。结果表明本文的系统已经形成了模型训练、模型推理、状态判断、界面显示的闭环。需要说明的是，系统演示仍然采用图片、视频输入的方式，摄像头接口虽然保留了，但是由于本机测试环境的限制，没有进行长时间的实机验证，在第 6 章功能测试、第 7 章展望中会对此进行说明。

5.6 本章小结

本章完成了实验环境、评价指标、对比结果、曲线、混淆矩阵和系统演示分析。结果表明，CBAM 提升了 mAP50、mAP50-95 和 Recall，并改善了 Yawn 类别表现，但仍存在误检和复杂场景不足问题。这些结果为系统实现和后续改进提供依据。

第 6 章 系统设计与实现

在完成实验验证后，本章说明系统实现过程，包括需求分析、总体架构、功能模块、运行环境、推理流程和桌面界面。

6.1 系统需求分析

本文主要是对毕业设计演示、疲劳驾驶面部状态识别实验等场景中训练得到的模型权重进行转化，得到可以操作、可以观察到的软件功能。系统应具有模型加载功能、图片输入功能、视频输入功能、检测启动功能、检测停止功能、检测结果显示功能、疲劳状态提示功能、截图保存功能、日志记录功能。对于摄像头输入，系统保留了接口以及处理流程，但是由于本机测试环境的限制，在本次没有进行稳定实机的验证。

从非功能需求上看，系统应该尽可能轻量、可复现、易调试。模型推理不会影响到界面的操作，检测结果包括类别名称、置信度、边界框，运行日志会显示输入的路径、模型的路径以及出现的问题。由于本文主要对毕业设计的实验验证进行系统的设计，所以系统更加注重功能闭环以及可解释性地表现，而并非是复杂的车载部署。

6.2 系统总体架构设计

系统的总体结构如图 6-1 所示。系统输入层接收到的是图片、视频或者摄像头帧，预处理层会对这些图像进行尺寸缩放、归一化、补全操作，模型推理层会加载 YOLOv8n 加 CBAM 的权重去预测出每一个检测框的坐标位置、分类标签以及置信度分数，疲劳判定层会利用一个固定大小的窗口来计算每一个检测框中的闭眼概率以及打哈欠次数，然后给出正常、疑似疲劳或者疲劳三个等级的结果，最后是界面展示层，它用来绘制出结果、显示状态、保存截图和记录日志。



图 6-1 疲劳驾驶面部识别系统总体架构

该架构是算法模块与界面模块分离的。模型训练模块得到权重，推理检测模块处理输入帧，疲劳判定模块把检测结果转换成状态提示，桌面演示模块和用户交互，显示结果。模块化的结构使得对于模型的权重重新设置，对于阈值的疲劳检测参数修改等更改起来比较简单。系统功能模块划分如表 6-1 所示。

6.3 功能模块设计

表 6-1 系统功能模块划分

模块	主要功能	论文作用
数据管理模块	维护数据集、类别顺序、训练配置和实验结果	保证训练数据和论文统计一致
模型训练模块	训练 YOLOv8n 与 YOLOv8n + CBAM 模型	支撑注意力机制对比实验
推理检测模块	处理图片、视频和摄像头输入，输出检测框与置信度	实现算法到系统的连接
疲劳判定模块	根据闭眼比例和打哈欠次数输出状态	将检测目标转化为可理解状态
桌面演示模块	显示检测结果、状态、截图和日志路径	完成可视化演示和结果留存

6.4 开发环境与项目结构

系统采用 Python 作为主要开发语言，深度学习框架使用 PyTorch，目标检测部分基于 Ultralytics YOLOv8 实现，图像处理使用 OpenCV，桌面界面使用 PyQt5。服务器端用 Slurm GPU 节点做训练，本地 Windows 环境主要是做模型推理和桌面演示。

项目结构以配置、训练、推理、界面、工具、权重、结果为序。configs 目录保存数据集配置和模型结构配置，src/train 保存训练入口，src/infer 保存推理入口，src/app 保存 PyQt5 桌面程序，src/utils 保存疲劳判定规则和 Ultralytics 兼容补丁，weights 目录保存训练权重，runs 目录保存训练日志、曲线图、检测结果和截图。因此模型实验和系统演示互不干扰，在模型实验完成的情况下再进行系统的演示，可以找到问题，也可以进行实际的操作。

6.5 模型训练与推理模块实现

模型训练入口对于 Ultralytics YOLO 的训练接口做了封装，可以设置数据集配置、模型配置、训练轮数、输入尺寸、batch size、workers 数量、设备序号等参数。CBAM 改进模型通过读取 configs/yolov8n_cbam.yaml 构建网络结构，并从 YOLOv8n 预训练权重初始化。训练结束之后，系统会保存 best.pt 等权重文件，并且还会输出指标表、PR 曲线、F1 曲线以及混淆矩阵等，这些数据可以供第五章进行分析使用。

推理模块是对图片、视频或者摄像头帧使用已训练好的模型权重来实现的。每次推理之后，模块就解析检测框坐标、类别编号、类别名称和置信度，然后调用 OpenCV 把检测框和文字画到图像上。视频输入逐帧进行处理，经过疲劳检测规则判断后的检测结果用在图片上。

在系统实现过程中还要考虑界面响应和模型推理之间存在着一种关系。当图片较大或者视频帧较多的时候，如果直接在主线程中执行模型预测的话，那么界面就会出现卡顿的情况。因此系统设计时把输入读取、模型推理、结果绘制以及状态更新分成比较独立的步骤。界面只起触发任务、显示结果的作用，主要的检测逻辑是由推理模块来完成的。这样既保持了代码结构的清晰，又方便了后面替换模型权重或者扩展视频输入的功能。

疲劳判定模块和第 4 章的规则一样。模块维护一个长度为 30 帧的滑动窗口，统计窗口内闭眼帧数、打哈欠次数，根据闭眼比例、打哈欠次数输出状态。为了防止由于某个帧出现误判而使整个报警机制被触发，连续三次闭眼后，连续打哈欠，连续三个时间点都被识别到之后，会将连续疲劳信号变成疲劳提醒。

6.6 桌面系统实现与测试

桌面系统入口用 PyQt5 主窗口来实现。界面是由模型选择、输入设置、开始检测、停止

检测、保存截图、显示日志这几项所组成的一层架构。为了避免模型推理造成的界面卡顿，在视频检测的过程中，把视频检测的工作交给单独的一个线程来完成，线程通过信号槽向主界面传入图像帧、检测结果、日志信息等数据。主界面主要起到显示、交互的作用，并不直接执行耗时的推理运算。

功能测试是对模型加载、图片检测、视频检测、疲劳状态显示、截图保存、日志记录和摄像头接口等进行测试的结果如表 6-2 所示。测试结果显示模型加载功能、图片检测功能、视频检测功能、疲劳状态显示功能、日志记录功能、截图保存功能均可以正常工作。摄像头输入功能已经保留接口，但是由于本机硬件和测试环境的限制，没有进行稳定的实机验证，因此在表 6-2 中标记为“接口保留，待实机验证”，不再表述为已经完全通过。

本文从系统的角度实现了毕业设计要求的最基本的功能闭环，模型权重可以被加载、输入的图像可以检测、检测结果可以转换为疲劳级别并在界面上进行展示。系统存在的不足有摄像头实时输入、长时间运行稳定性、报警交互设计等各方面，在真实设备、真实驾驶环境的测试中还应继续进行。

表 6-2 系统功能测试结果

功能	测试内容	测试结果
模型加载	选择训练得到的 best_cbam.pt 权重并初始化模型	通过
图片检测	输入单张图片并显示检测框、类别和置信度	通过
视频检测	读取视频帧并逐帧输出检测结果	通过
疲劳状态显示	根据闭眼比例和打哈欠次数输出正常/疑似疲劳/疲劳	通过
截图保存	保存当前检测画面到指定路径	通过
日志记录	记录模型路径、输入路径、状态提示和异常信息	通过
摄像头输入	保留摄像头采集接口，待真实设备环境验证	接口保留，待实机验证

6.7 本章小结

本章完成了系统需求、总体架构、功能模块、运行环境、推理流程、桌面界面和功能测试说明。系统将模型检测结果转化为可演示的软件功能，形成训练、推理和展示的闭环，并为第七章总结与展望提供基础。

第 7 章 总结与展望

7.1 结论

本文就疲劳驾驶面部识别任务提出了一种使用 YOLOv8 以及注意力机制的检测系统。首先整理出疲劳驾驶检测相关的背景技术基础，然后创建闭眼、睁眼、打哈欠三个面部目标的训练数据集，就分类分布、标注格式、数据增强、数据集不足这四个部分做分析。

本文用 YOLOv8n 做为基线模型，用 CBAM 注意力机制来构建 YOLOv8n+CBAM 改进模型。实验结果表明，改进模型 mAP50 由 0.8347 提高到 0.8580，mAP50-95 由 0.5368 提高到 0.5470，Recall 由 0.9114 提高到 0.9370。Yawn 类别 mAP50 从原来的 0.693 提高到了现在的 0.762，说明注意力机制对于嘴部局部疲劳特征有一定程度的加强。本文用 PyQt5 来实现桌面演示界面。系统可以对检测结果进行显示、疲劳程度提醒、截屏保存、日志记录等操作来构成一个闭环，即检测-推理-分析-展示的闭环。

本文所完成的工作具有一定的综合性。数据集部分解决的是训练样本的组织、类别标注问题。模型部分研究了改进局部疲劳现象的 CBAM 的改良。实验部分给出了改进模型对于 mAP50、Recall 和 Yawn 类别的效果。系统部分把检测的结果变成可以被人们所看见的状态，也就是检测结果对应的报警信息。目前系统还存在着数据量大、实时视频测试以及复杂驾驶环境下模型训练、实验分析和原型展示等各方面的问題，但是能够满足毕业设计的要求。因此后续修改的重点应该继续从数据的真实性、实时稳定性、系统可用性三个方面入手，使论文中所得到的实验结果同工程实现有更加紧密的对应关系。也给后面继续完善论文和系统提供清楚的依据。论文的结构更严密了。章节之间更加自然。更为充分。更加完整。

7.2 创新点与局限性

本文的创新点主要有三个方面。第一，用 YOLOv8n 轻量级目标检测模型对闭眼、睁眼、打哈欠这三种可以解释的面部目标进行检测，使得疲劳状态的判定有了可以观察到的中间证据。第二，在基线模型的基础上加入 CBAM 注意力机制，用两组对比实验来分析它对总体指标以及 Yawn 类别的影响。第三部分就是把模型训练、推理检测、桌面演示系统这三者联系起来，形成一个可以完成毕业设计展示、实验复现的工程闭环。

本文也有一定的不足之处。数据集主要是从公开整理的数据中获得的，对于复杂驾驶环境下夜间弱光、强光反射、遮挡、侧脸以及真实车载摄像头样本仍然比较缺乏；其次，疲劳状态判断主要是依靠单帧检测结果和滑动窗口规则，还没有很好地建立长时间连续疲劳趋势的模型；再次，摄像头实时检测 and 不同硬件环境下的稳定性测试还不充分，距离真实驾驶辅助系统还有一定的差距。

7.3 后续展望

后续工作可以从以下几个方面进行。保留原数据集中所有的图像，并在原有基础上加上夜景照片、遮挡的部分、多个角度的照片、不同的镜头角度等新的图片来增加模型理解各种情况的能力。第二，用连续帧特征去提取出 LSTM、Transformer 这些时序模型的输入特征后，再进行训练得到更加稳定疲劳状态的判断结果。第三部分是对模型的轻量化、量化的压缩及在边端部署的效果进行评估，在不同的硬件上做实时推理性能的实验研究。第四，根据前面的理论对现有的系统中的报警策略、交互方式等做进一步的改善，在不降低系统性能的情况下更加贴近真实车辆辅助驾驶的功能需求。

从数据角度来讲，以后要加强真实驾驶场景数据的收集。目前的数据集可以进行闭眼、睁眼和打哈欠的检测训练，但是样本来源、摄像头的角度以及驾驶环境还比较单一。接着选取了夜间、逆光、戴眼镜、戴口罩、侧脸、低头、多人背景等样本来清洗异常标签。尤其是一些打哈欠的类别很容易和说话、张嘴等表情相混淆，因此应该加上困难负样本来降低误报率。

从模型上来说，满足轻量化要求的同时也可以比较 SE、ECA、Coordinate Attention 等注意力模块，在相同数据集下使用相同模型的情况下可以清楚地看出各个版本 YOLO 模型的精度、参数量、推理速度等各方面的情况，更好地体现出模型结构选择与实际部署需求的关系。

就系统而言，应该对视频流的测试和报警交互加以改进。真正意义上的自动驾驶系统并不只是检测前方有车辆，还要就其检测帧率、识别是否合理、有声音发出以及是否进行过用户输入进行相应的控制与记录。本文已经形成了一个基本的演示闭环，后面可以增加连续驾驶时间统计、疲劳事件回放、报警等级设置、模型热切换等功能。



参考文献

- [1] REDMON J, DIVVALA S, GIRSHICK R, et al. You only look once: Unified, real-time object detection[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2016: 779-788.
- [2] WOO S, PARK J, LEE J Y, et al. CBAM: Convolutional block attention module[C]//Proceedings of the European Conference on Computer Vision (ECCV). Cham: Springer, 2018: 3-19.
- [3] ALBADAWI Y, TAKRURI M, AWAD M. A review of recent developments in driver drowsiness detection systems[J]. Sensors, 2022, 22(5): 2069. DOI:10.3390/s22052069.
- [4] SALEEM A A, SIDDIQUI H U R, RAZA M A, et al. A systematic review of physiological signals based driver drowsiness detection systems[J]. Measurement, 2023, 216: 112980.
- [5] 罗晓璐. 基于 OpenCV 的驾驶员疲劳状态检测系统[J]. 信息技术与信息化, 2025, (09): 50-54.
- [6] 邹晓越. 基于机器视觉的驾驶员面部多特征疲劳检测系统研究[D]. 山东交通学院, 2025. DOI:10.27864/d.cnki.gsjtd.2025.000120.
- [7] 魏恒建. 基于多通道信息融合的驾驶员疲劳检测系统研究[D]. 大连交通大学, 2024. DOI:10.26990/d.cnki.gsltc.2024.000335.
- [8] 孙超. 基于计算机视觉与驾驶信息的驾驶安全检测方法研究[D]. 内蒙古大学, 2024. DOI:10.27224/d.cnki.gnmdu.2024.000896.
- [9] 丁子原. 基于改进 YOLOv8 的疲劳驾驶检测轻量化研究[D]. 大连交通大学, 2025. DOI:10.26990/d.cnki.gsltc.2025.000997.
- [10] 刘德馨. 基于面部特征的疲劳驾驶检测技术研究[D]. 电子科技大学, 2025. DOI:10.27005/d.cnki.gdzku.2025.000281.
- [11] 刘鹏宇. 基于驾驶员面部特征的疲劳驾驶检测算法研究与应用[D]. 太原师范学院, 2024. DOI:10.27844/d.cnki.gtysf.2024.000066.
- [12] 王丹卿. 基于面部特征与深度学习的疲劳检测算法的研究[D]. 江苏大学, 2023. DOI:10.27170/d.cnki.gjsuu.2023.001562.



- [13] JOCHER G, CHAURASIA A, QIU J. Ultralytics YOLOv8[EB/OL]. (2023-01-10)[2026-05-04]. <https://github.com/ultralytics/ultralytics>.
- [14] XIE S Y, CHUAH J H, CHAI G M T. Revolutionizing road safety: YOLOv8-powered driver fatigue detection[C]//2023 IEEE Asia-Pacific Conference on Computer Science and Data Engineering (CSDE). 2023. DOI:10.1109/CSDE59766.2023.10487765.
- [15] ZHOU C, ZHAO Y, LIU S B, et al. Research on driver facial fatigue detection based on YOLOv8 model[EB/OL]. arXiv:2406.18575, 2024. DOI:10.48550/arXiv.2406.18575.
- [16] LIU G C, WU K, LAN W, et al. YOLO-FDCL: improved YOLOv8 for driver fatigue detection in complex lighting conditions[J]. Sensors, 2025, 25(15): 4832. DOI:10.3390/s25154832.
- [17] PASZKE A, GROSS S, MASSA F, et al. PyTorch: an imperative style, high-performance deep learning library[C]//Advances in Neural Information Processing Systems (NeurIPS). 2019, 32: 8026-8037.
- [18] OPENCV TEAM. OpenCV 4.x documentation[EB/OL]. [2026-05-04]. <https://docs.opencv.org/4.x/>.
- [19] RIVERBANK COMPUTING. PyQt[EB/OL]. [2026-05-20]. <https://www.riverbankcomputing.com/software/pyqt>.



致谢

本文是在指导教师的悉心指导下完成的。从选题确定、方案论证、系统实现到论文写作，指导教师一直给予细致而有针对性的研究方向指导，在研究过程中遇到困难时也提出了许多宝贵建议，使研究工作能够顺利开展。能够在认真负责的学术氛围中完成本次毕业设计，我深感荣幸，并向指导教师表示诚挚感谢。

感谢黑龙江工业学院电气与信息工程学院提供学习平台和实验环境，也感谢专业课程教师在计算机视觉、深度学习和 Python 编程等课程中的指导，这些知识积累为本文完成打下了良好基础。感谢实验室同学和身边同学在学习、调试和资料交流过程中给予的帮助与支持。

本次毕业设计持续时间较长，环境搭建、数据集准备、模型训练和桌面系统开发等环节都经历了不同程度的调试和修正。通过本次设计，我进一步理解了 YOLOv8 目标检测框架和 CBAM 注意力机制，也提升了独立分析问题、解决工程问题和撰写论文的能力。谨向所有给予帮助和支持的老师、同学及家人表示感谢。



附录1 设计流程图

(放入一张流程图)

附录2 部分源程序

以下为系统核心源程序节选，主要包括疲劳状态规则判断、YOLOv8 单帧推理和桌面演示系统帧处理代码。

```
# 1. 疲劳状态规则判断核心代码 (src/utils/fatigue_rules.py)
from collections import deque
from dataclasses import dataclass
from typing import Deque, Dict, Iterable, Tuple

CLASS_NAME_ALIASES = {
    "eyeopen": "eye_open",
    "eye_open": "eye_open",
    "eyeclosed": "eye_closed",
    "eye_closed": "eye_closed",
    "yawn": "yawn",
    "yawning": "yawn",
}

@dataclass
class FrameFatigueFeature:
    """单帧疲劳特征，用于保存睁眼、闭眼和打哈欠目标的最高置信度。"""

    eye_open_conf: float = 0.0
    eye_closed_conf: float = 0.0
    yawn_conf: float = 0.0

    def as_tuple(self) -> Tuple[float, float, float]:
        return (self.eye_open_conf, self.eye_closed_conf, self.yawn_conf)

class FatigueRuleEvaluator:
    """基于滑动窗口的疲劳状态判断器。"""

    def __init__(
        self,
        window_size: int = 90,
        eye_closed_threshold: float = 0.45,
        yawn_threshold: float = 0.50,
        fatigue_ratio: float = 0.35,
        suspected_ratio: float = 0.20,
    ) -> None:
        self.window_size = window_size
```




```
self.eye_closed_threshold = eye_closed_threshold
self.yawn_threshold = yawn_threshold
self.fatigue_ratio = fatigue_ratio
self.suspected_ratio = suspected_ratio
self._features: Deque[FrameFatigueFeature] = deque(maxlen=window_size)

def update(self, feature: FrameFatigueFeature) -> Dict[str, float | str | bool]:
    self._features.append(feature)
    return self.evaluate()

def evaluate(self) -> Dict[str, float | str | bool]:
    if not self._features:
        return {"state": "normal", "closed_ratio": 0.0, "yawn_count": 0, "alarm": False}

    closed_flags = [
        item.eye_closed_conf >= self.eye_closed_threshold
        and item.eye_closed_conf >= item.eye_open_conf
        for item in self._features
    ]
    yawn_flags = [item.yawn_conf >= self.yawn_threshold for item in self._features]
    closed_ratio = sum(closed_flags) / len(closed_flags)
    yawn_count = self._count_events(yawn_flags)

    if closed_ratio >= self.fatigue_ratio or yawn_count >= 2:
        state = "fatigue"
    elif closed_ratio >= self.suspected_ratio or yawn_count >= 1:
        state = "suspected_fatigue"
    else:
        state = "normal"

    return {
        "state": state,
        "closed_ratio": round(closed_ratio, 4),
        "yawn_count": yawn_count,
        "alarm": state == "fatigue",
    }

@staticmethod
def _count_events(flags: Iterable[bool]) -> int:
    count = 0
    in_event = False
    for flag in flags:
        if flag and not in_event:
            count += 1
            in_event = True
        elif not flag:
            in_event = False
    return count
```



```
def feature_from_detections(detections: Iterable[Tuple[str, float]]) -> FrameFatigueFeature:
    """从 YOLO 检测结果中提取单帧疲劳特征。"""

    feature = FrameFatigueFeature()
    for class_name, confidence in detections:
        normalized_name = CLASS_NAME_ALIASES.get(
            class_name.strip().lower().replace("-", "_").replace(" ", "_")
        )
        conf = float(confidence)
        if normalized_name == "eye_open":
            feature.eye_open_conf = max(feature.eye_open_conf, conf)
        elif normalized_name == "eye_closed":
            feature.eye_closed_conf = max(feature.eye_closed_conf, conf)
        elif normalized_name == "yawn":
            feature.yawn_conf = max(feature.yawn_conf, conf)
    return feature


# 2. 单帧推理与状态叠加代码 (src/infer/run_infer.py)
def detections_from_result(result):
    """从 Ultralytics 单帧结果中提取类别名和置信度。"""

    detections = []
    names = result.names
    for box in result.bboxes:
        class_id = int(box.cls[0].item())
        confidence = float(box.conf[0].item())
        detections.append((str(names[class_id]), confidence))
    return detections


def draw_status(frame, state_info: dict, fps: float):
    """在检测画面左上角绘制 FPS、疲劳状态和闭眼比例。"""

    state = str(state_info["state"])
    alarm = bool(state_info["alarm"])
    color = (0, 0, 255) if alarm else (0, 180, 255) if state == "suspected_fatigue" else (0, 180, 0)
    text = f"State: {state} | FPS: {fps:.1f} | Closed: {state_info['closed_ratio']}"
    cv2.rectangle(frame, (8, 8), (780, 48), (0, 0, 0), thickness=-1)
    cv2.putText(frame, text, (18, 36), cv2.FONT_HERSHEY_SIMPLEX, 0.75, color, 2, cv2.LINE_AA)
    return frame


def run_frame_infer(model, frame, evaluator: FatigueRuleEvaluator, conf: float):
    """对单帧图像执行 YOLOv8 检测，并输出疲劳状态。"""

    start = time.perf_counter()
    result = model.predict(frame, conf=conf, verbose=False)[0]
```



```

annotated = result.plot()
detections = detections_from_result(result)
feature = feature_from_detections(detections)
state_info = evaluator.update(feature)
fps = 1.0 / max(time.perf_counter() - start, 1e-6)
return draw_status(annotated, state_info, fps), detections, state_info, fps

```

3. 桌面演示系统帧处理代码 (src/app/main_window.py)

class VideoWorker(QThread):

"""视频检测线程，避免模型推理阻塞 PyQt 主界面。"""

frame_ready = pyqtSignal(object)

status_ready = pyqtSignal(str, bool, str)

log_path_ready = pyqtSignal(str)

error_ready = pyqtSignal(str)

def _process_frame(self, frame, log_writer) -> None:

"""处理一帧图像，并把检测画面、状态文本和日志记录发送给界面。"""

start = time.perf_counter()

result = self.model.predict(frame, conf=self.conf, verbose=False)[0]

annotated = result.plot()

detections = detections_from_result(result)

feature = feature_from_detections(detections)

state_info = self.evaluator.update(feature)

fps = 1.0 / max(time.perf_counter() - start, 1e-6)

state = str(state_info["state"])

state_cn = STATE_TEXT.get(state, state)

detection_text = "; ".join(f"{name}:{conf:.2f}" for name, conf in detections) or "未检测到

目标"

state_text = (

f"状态: {state_cn} "

f"闭眼比例: {state_info['closed_ratio']} "

f"打哈欠次数: {state_info['yawn_count']} "

f"FPS: {fps:.1f}"

)

if log_writer is not None:

log_writer.writerow(

[

self.frame_index,

state,

state_info["closed_ratio"],

state_info["yawn_count"],

int(bool(state_info["alarm"])),

round(fps, 2),

detection_text,

]



```
)  
self.status_ready.emit(state_text, bool(state_info["alarm"]), detection_text)  
self.frame_ready.emit(annotated)  
self.frame_index += 1
```